YOG MILK

B2B Dairy Ordering & Retailer Management Platform

Master Planning Document | Phase 1: Planning & Requirement Analysis | Phase 2: Development Execution & Implementation Planning

Version: 2.0 — Revised requirements | Prepared: 29 October 2026

This document supersedes earlier planning that included retailer credit, credit expiry, credit-based orders, and seven-day credit payment terms. Those features are removed from the approved scope described here.

Document Purpose

This document is the working source of truth for the Yog Milk project. It defines the product scope, roles, business rules, user experiences, workflows, notification expectations, database architecture, security boundaries, delivery and payment handling, billing requirements, and the staged development plan. It is intended for the project owner, client, designer, developer, and coding agent.

Where the client has made a clear decision, this document treats it as a requirement. Where a detail is not yet specified, it is marked as a decision or implementation detail to confirm rather than silently assumed.

1. Project Overview

1.1 Business objective

Build a reliable digital ordering and operations platform for Yog Milk, a dairy business supplying products in bulk to retailers and other approved business customers. Retailers will place orders through an Android app. Yog Milk staff will operate orders, payments, delivery updates, products, retailers, schedules, invoices, notifications, and reports through a responsive web dashboard.

1.2 Approved platform and technology direction

- Retailer application: Flutter and Dart for Android.
- Admin panel: responsive web dashboard, deployed on Vercel.
- Backend/database: Supabase with PostgreSQL.
- Authentication: Google Sign-In connected to Supabase Auth.
- Online payment gateway: Razorpay.
- Languages: English, Hindi, and Marathi.
- Push notifications: Firebase Cloud Messaging (FCM), supported by persistent in-app notifications.
- Database realtime updates: Supabase Realtime where appropriate, backed by durable notification records.
- Recommended Flutter structure: feature-based modular architecture, Riverpod for state management, GoRouter for navigation, and a consistent Material design system.
- Recommended admin web structure: a maintained React/Next.js-based dashboard deployed to Vercel; exact framework version and component library to be selected at implementation kickoff.

1.3 Main deliverables

1. Retailer Android application.

2. Admin web dashboard.
3. Supabase database, authentication, access controls, storage, and backend functions.
4. Razorpay payment integration and cash-on-delivery/offline payment recording.
5. Order and delivery status management with unique references.
6. Notification center for retailers and administrators.
7. Non-GST invoice/bill generation based on the client's current stated business setup.
8. Retailer 360° profile, transaction history, analytics, reports, and audit trail.
9. Production testing, deployment, documentation, and handover.

PHASE 1 — PLANNING & REQUIREMENT ANALYSIS

2. Business Scope and Core Rules

2.1 In-scope capabilities

- Retailer registration/sign-in and account approval or activation.
- Product catalogue, categories, images, availability, pack sizes, wholesale/business pricing, and quantity rules.
- Bulk cart, checkout, quick add, frequently ordered products, and reorder from past orders.
- Admin-configurable ordering schedule: define permitted intervals and days, with automatic opening/closing and announcements for schedule changes.
- Orders with unique human-readable order codes and persistent order history.
- Separate payment status and delivery status, with individual admin updates where required.
- Online payment through Razorpay and Cash on Delivery (COD).
- COD/offline payment status remains pending until an authorized admin records the payment as received.
- Payment-pending order restriction: a retailer cannot place another order while an earlier order has an unpaid/pending payment, according to the approved blocking rule.
- Delivery status updates entered by an administrator, with a unique delivery reference/code for each order delivery.
- Retailer and admin notifications for meaningful business events, including orders, payment updates, delivery changes, product arrivals/availability, and order-window opening/closing.
- Non-GST commercial billing/invoices and downloadable PDFs, with accurate order totals and payment status.
- Retailer history, payment ledger, invoices, delivery timeline, purchasing analytics, and admin audit logs.
- Admin reports, dashboard metrics, searchable/filterable records, and notification controls.
- English, Hindi, and Marathi UI localization.

2.2 Explicitly removed from scope

- Credit allocation or credit percentage earned from purchases.
- Credit balance, credit wallet, credit ledger, credit usage, and credit expiry.
- Orders placed using credit or split credit/payment calculations.
- Seven-day credit payment terms, credit due dates, credit-expiry reminders, and credit-specific restrictions.
- Any dashboard, notification, database table, UI screen, analytics metric, or business rule whose purpose is managing retailer credit.

Implementation rule: do not create credit-related tables or UI placeholders “for later” in the initial implementation. If the client requests credit in the future, it must be treated as a separately approved scope change with a new design and migration plan.

2.3 Roles

Retailer

- Sign in using Google and complete business/shop profile information.
- Browse available products and current prices.
- Place bulk orders only while ordering is open and the account is eligible.
- Choose Razorpay online payment or Cash on Delivery where enabled.
- See order, payment, invoice, and delivery statuses.
- View order history, payment history, invoices, delivery updates, and personal purchase analytics.
- Receive notifications for relevant Yog Milk updates.
- Change app language among English, Hindi, and Marathi.

Administrator / authorized staff

- Manage retailer accounts and activation status.
- Manage products, categories, images, availability, prices, and quantity rules.
- Configure ordering hours, applicable days, and scheduled exceptions; the system automatically determines whether ordering is open or closed.
- View all orders and update each order's status individually.
- View payment records and verify/record offline or COD payments with an audit trail.
- Manage delivery status individually and maintain delivery notes/reference codes.
- Generate, view, download, and manage invoices according to approved rules.
- View complete retailer transaction history and analytics.
- Receive operational notifications and send system-wide or targeted announcements.
- Review audit history and export authorized reports.

Administrative access must be role-based. The first release may use one owner/admin role, but the data model should allow least-privilege roles later, such as Operations, Accounts, and Order Manager. A retailer must never be able to access another retailer's records.

3. Retailer App — Functional Requirements

3.1 Sign-in and retailer onboarding

- Google Sign-In establishes the identity through Supabase Auth.
- Collect required business fields such as shop/business name, owner's name, contact number, email if available, business address, delivery address, and account status.
- Google authentication alone must not be treated as proof of business eligibility.
- Where the client requires approval, new accounts remain Pending Approval until an admin activates them.
- Inactive, suspended, or unapproved accounts cannot submit orders; the app should explain the reason.

3.2 Home

- Retailer greeting and shop name.

- Current order-window status: OPEN or CLOSED, with opening/closing information where available.
- Quick Add and frequently purchased products.
- Reorder Last Order.
- Current orders and their latest statuses.
- Pending payment notice, when applicable.
- Recent invoices and recent notifications.
- Product arrival/availability announcements.

3.3 Catalogue and pricing

- Browse/search/filter products by category.
- Show product name, image, pack size, unit, price, availability, minimum quantity, and quantity increment rules.
- Allow admin-configured retailer-specific or wholesale pricing if required by Yog Milk.
- Show unavailable products clearly; do not accept them at checkout.
- Backend recalculates order totals using trusted current prices; client-submitted prices are never trusted.

3.4 Cart and checkout

- Add multiple products and quantities to one bulk order.
- Show line totals, subtotal, approved discounts if any, delivery charge if configured, final total, payment method, and delivery address.
- Validate order window, account status, product availability, quantity rules, current prices, and unpaid-order restriction before accepting the order.
- Recalculate totals server-side at submission.
- Show an order summary and unique order code after successful order creation.
- Prevent a cart created before closing time from bypassing the backend ordering-window rule.

If the current time is outside the administrator-configured ordering interval, the retailer may continue browsing the catalogue and using the app, but any attempt to submit an order (including Buy Now) must be blocked. Show a clear message such as: “The ordering window is closed. We’ll notify you when ordering opens again.” Do not create an order from this attempt. If the retailer has enabled ordering notifications, notify them when the next configured ordering interval begins.

3.5 Quick Add and reorder

Retailers can add quantities of frequently ordered products from a compact screen and rebuild a new cart from a previous order. Reorder must create a new order and must use current prices and current product availability; it must not edit the historical order.

3.6 Orders and order details

- List current, processing, ready, out-for-delivery, delivered, cancelled, and failed orders where applicable.
- Each order detail shows unique order code, creation time, items, quantities, price snapshot, totals, payment method/status, invoice, delivery reference, and chronological status history.
- Order status and payment status must be displayed separately.
- Retailer can see updates after the admin changes a status.

3.7 Payments

- Payment options: Razorpay online payment and COD, subject to admin configuration.

- Online payment: show the gateway result only after trusted backend verification.
- COD: order is created with payment status PENDING; the retailer must not be shown as paid until authorized admin confirmation is recorded.
- Display amount due, payment method, transaction/reference details, payment date, and current status.
- Display failed online attempts without overwriting prior successful transactions.
- Where an unpaid/pending payment blocks new orders, show a clear blocking message and a link to the unpaid order/payment details.

3.8 Delivery

The initial release uses admin-managed delivery updates. Retailers see the latest delivery status, delivery code/reference, timestamps, and any customer-visible note. Live GPS tracking and a separate driver application are not included unless approved as a later phase.

3.9 Billing and invoice access

- Retailer can open invoices from the order detail and invoice history.
- Invoice PDF can be viewed/downloaded and shared using an authorized link or file-access mechanism.
- Invoice includes unique invoice number, date, Yog Milk business details, retailer details, order code, product lines, quantities, prices, total, payment status, amount paid, and amount due.
- Current requirement is a non-GST commercial invoice; do not print GSTIN, GST tax rates, CGST/SGST/IGST, or label it a GST tax invoice unless the business's legal/tax status changes and the client approves the change.

3.10 Retailer history and analytics

- Complete order history and product-wise purchase history.
- Monthly purchase totals and order counts.
- Most frequently purchased products and recent orders.
- Payments made, pending payments, failed attempts, and refunds where applicable.
- Delivery history and invoice history.
- All values are derived from backend records.

4. Admin Web Dashboard — Functional Requirements

4.1 Dashboard

- Orders today, orders awaiting action, orders processing, deliveries due, delivered orders, active retailers, sales/order value, payments collected, and pending payment amount.
- Separate order value from actual collected payments and unpaid balances; these are different measures.
- Operational alerts for new orders, pending COD collection, unpaid-order blocks, failed online payments, delivery issues, and schedule changes.
- Filters by date range and export of permitted reports.

4.2 Retailer management and Retailer 360°

An admin can search retailers and open a complete profile containing business/contact details, account status, orders, payments, invoices, deliveries, purchase analytics, notifications relevant to the retailer, and a chronological activity timeline.

- Summary metrics: total orders, total billed/order value, total payments received, pending amount, delivered orders, last order, last payment, and frequently purchased products.
- Search/filter all orders and payments for the retailer.
- Review every relevant status change and who performed it.
- Activate, suspend, or update permitted profile fields with a reason where appropriate.

4.3 Product and pricing management

- Create/edit/deactivate products and categories.
- Upload product images and manage pack size, unit, description, availability, minimum order quantity, increments, and prices.
- Optionally support retailer-specific pricing if the business confirms it.
- Product arrival/new stock can be announced to retailers, either automatically when availability is changed or through an admin announcement.
- Historical order items preserve the name, unit price, quantity, and line totals that applied at order time.

4.4 Order management

- Search and filter orders by order code, retailer, date, order status, payment status, and delivery status.
- Open order detail with immutable order-item snapshot, totals, retailer information, payment records, delivery details, invoice, and status timeline.
- Update order status individually; do not automatically mark delivered based only on elapsed time.
- Require valid state transitions, confirmation prompts for sensitive actions, and reason notes for exceptions/cancellations.
- Every status change records previous status, new status, actor/admin ID, timestamp, and optional reason.

4.5 Payment operations

- View all payment attempts and records, with filters for pending, paid, failed, refunded, and partially refunded where supported.
- Razorpay payments are confirmed through server-side signature/webhook verification, not by trusting a browser or app success screen.
- COD/offline collection is marked PAID only by an authorized admin after the money has actually been received.
- Manual payment entry requires amount, method, date/time, reference/receipt if available, and a reason/note; it must create an auditable record.
- Do not delete or overwrite prior payment attempts; preserve transaction history and use reversal/refund records when needed.
- Admin can view which retailer accounts are blocked from ordering because of pending payments.

4.6 Delivery management

- Queue of orders needing delivery preparation, assignment or dispatch, and completion.
- Admin updates each delivery's status individually.
- Unique delivery code/reference linked to the unique order code.
- Capture status timestamp, admin/staff actor, notes, and failure reason if applicable.
- Delivery status changes create retailer and admin notifications according to event rules.

4.7 Ordering schedule settings

- Configurable recurring ordering intervals; ordering opens and closes automatically according to the saved schedule.
- Editable opening and closing times, with the ability for an administrator to change the schedule whenever business hours change.
- Applicable days of the week and scheduled exception/holiday closure dates, if required.
- Time zone fixed to the business's configured local zone (expected Asia/Kolkata).
- The next opening time and ordering-closed message shown to retailers when ordering is outside the configured interval.
- Every schedule change is audited. Notify relevant retailers about the updated schedule and the next opening time where applicable.

4.8 Billing and reports

- Invoice list, search, retailer/date filters, payment status, PDF preview/download, and outstanding invoice report.
- Invoice totals are tied to the source order and its item/price snapshots.
- Payment records are separate from invoices; payments update amount paid and outstanding balance without rewriting the original invoice amount.
- Export orders, payments, deliveries, retailer history, and sales/collection summaries where authorized.

4.9 Admin notification center

The admin dashboard must have a notification center showing new orders, payment state changes, pending COD collection, Razorpay failures, retailer order blocks, delivery status changes, newly available products, order-window changes, and system/operational alerts. Admins should be able to mark notifications read and navigate to the affected record.

5. Ordering Schedule and Order-Blocking Rules

5.1 Admin controls

Ordering must operate automatically from the administrator-configured schedule. Do not provide a master open/closed switch or a manual pause/resume control. The administrator can edit the opening time, closing time, applicable days, and scheduled exceptions; saved changes must take effect according to the updated schedule. All new-order submission paths, including Buy Now and checkout, must be blocked by the backend whenever the current time falls outside the permitted interval.

The administrator configures the recurring ordering interval, including opening/closing times and applicable weekdays. For example, ordering may run from 7:00 AM to 5:00 PM, but these are illustrative values only. The schedule must run automatically, and the administrator must be able to change its times or days whenever required. Any exception or holiday closure must be represented as a scheduled configuration, not as a master switch.

5.2 Enforcement

The backend—not only the app UI—must validate that ordering is open at the defined order-acceptance moment. Schedule checks must use the configured business timezone. The system must clearly define the acceptance moment and apply it consistently.

5.3 One pending-payment restriction

Business rule requested: if a retailer has an order whose payment is still pending, the retailer cannot place another new order until that pending payment is resolved. This restriction applies regardless of whether the unpaid order was placed using COD or another approved payment method, unless Yog Milk explicitly configures an exception.

Recommended precise rule: block new order submission while the retailer has at least one order with a positive amount due and a payment state that is not resolved. A verified Razorpay payment or an authorized admin-recorded COD/offline payment can resolve the block. A failed Razorpay attempt alone should not mark the order paid. If an order is cancelled, the backend must resolve its balance according to the cancellation/refund rules before releasing the block.

The app should show the blocking order code, pending amount, payment method, and next action. Admin can see the same block reason in the retailer profile and dashboard.

6. Order, Payment and Delivery Workflows

6.1 Order lifecycle

Recommended normal order progression: PLACED → CONFIRMED → PROCESSING → READY_FOR_DELIVERY → OUT_FOR_DELIVERY → DELIVERED. If Yog Milk prefers the order to enter PROCESSING immediately on placement, the system can display PLACED and transition to PROCESSING automatically after successful order creation; this behavior must be explicitly approved. The initial status should never be ambiguous.

Additional outcomes may include CANCELLED and DELIVERY_FAILED. State transitions must be validated by backend logic. An administrator can make only permitted transitions unless an exception permission and reason are provided.

6.2 Payment lifecycle

- PENDING — no confirmed payment recorded for the payable amount.
- PROCESSING — an online payment attempt is in progress, where applicable.
- PAID — the full amount has been received and verified.
- FAILED — a specific payment attempt failed; this does not erase another successful attempt.
- PARTIALLY_PAID — only if partial payment is explicitly enabled by the client; otherwise omit from the initial UI.
- REFUNDED / PARTIALLY_REFUNDED — where a refund has actually been processed and recorded.

Keep order status, payment status, and delivery status separate. An order can be PROCESSING while payment is PENDING and delivery is NOT_ASSIGNED.

6.3 COD flow

10. Retailer selects Cash on Delivery at checkout, if COD is enabled.
11. Backend validates the retailer, schedule, cart, pricing, and pending-payment restriction, then creates the order and unique order code.
12. Order status follows the agreed workflow (PLACED then PROCESSING if immediate processing is enabled); payment status is PENDING and method is COD.
13. Retailer and admin receive order-created notifications. Admin sees the COD collection as pending.
14. Authorized admin records payment received only after cash/offline collection; amount, timestamp, method, reference/note, and actor are stored.

15. Payment status changes to PAID only after the admin action is saved successfully.
16. Both retailer and admin are notified; the pending-payment restriction is removed only after backend confirms the payable balance is resolved.

6.4 Razorpay flow

17. Backend creates the payable order and Razorpay order using trusted server-side credentials.
18. Retailer completes payment using the supported gateway UI.
19. Backend verifies the payment signature and processes the provider webhook with idempotency protection.
20. Only verified successful payment events update the payment ledger and resolve the pending balance.
21. Retailer and admin receive payment confirmation; failed/pending attempts create appropriate notifications but do not falsely mark the order paid.

6.5 Delivery flow

22. Admin prepares the order and sets delivery status/assignment information.
23. Each delivery has its own unique delivery reference and is linked to the order.
24. Admin changes delivery status one step at a time (for example NOT_ASSIGNED → ASSIGNED → PICKED_UP → OUT_FOR_DELIVERY → DELIVERED).
25. Every change creates a history record with actor, timestamp, previous/new status, and optional note.
26. Retailer receives the update and can view the delivery timeline.
27. Failed delivery requires a reason and a defined follow-up action.

6.6 Unique codes

Every order must have an immutable database ID and a human-readable unique order code (for example, YM-2026-000123). Every delivery must have its own unique delivery code/reference linked to the order. Invoices also require their own unique invoice number. Examples are illustrative; the actual format and numbering rules should be agreed before production. Codes must be generated server-side and protected by database uniqueness constraints.

7. Notification System — Cross-Platform Requirement

7.1 Core principle

Every meaningful business update that the retailer or admin needs to act on or understand must create a durable notification/event. Notifications should not be limited to the screen currently open. Use in-app notifications plus push notifications where available, with realtime refresh for active sessions. Avoid sending a push notification for every low-level database write or technical field change; notify on business events and status transitions, with configurable noise controls.

7.2 Recipient rules

- Retailer receives events concerning their own account, orders, payments, deliveries, invoices, relevant product announcements, and ordering schedule.
- Admin receives operational events across the business: new orders, payment changes, pending COD collections, failed payments, blocked retailers, delivery updates/failures, product availability changes, and ordering schedule changes.

- A retailer must never receive another retailer's private order, payment, invoice, or profile information.
- If there are multiple admin roles, notification routing must follow permissions and operational responsibility.

7.3 Required notification events

- New order placed: notify the relevant retailer with confirmation and notify admin with retailer, order code, amount, and payment method.
- Order status changed: notify retailer and admin/operations log when the status changes.
- Payment status changed: notify retailer and admin for successful payment, failed attempt, pending state, refund, or manual COD payment confirmation.
- COD payment pending: alert admin until collection is recorded; optionally show a pending-payment reminder to the retailer.
- Pending payment blocks ordering: notify the retailer at the attempted checkout and surface an admin alert.
- Delivery status changed: notify retailer and admin/operations history; include delivery code and current status.
- Product newly added or newly available/arrived: notify relevant retailers based on configured audience, not necessarily every retailer for every product edit.
- Ordering opened: when the next configured ordering interval begins, notify retailers who have enabled ordering notifications and show OPEN status in the app.
- Ordering closed: show CLOSED status and the next scheduled opening time if known. If a retailer attempts to order using Buy Now or checkout while the schedule is closed, display: “The ordering window is closed. We’ll notify you when ordering opens again.” Do not create an order. When the next configured ordering interval begins, notify retailers who have enabled ordering notifications.
- Ordering schedule changed or a scheduled exception/holiday closure applies: notify affected retailers according to their notification preferences.
- Invoice generated or corrected: notify retailer and admin/accounts.
- Retailer approved, suspended, or deactivated: notify the retailer with appropriate wording and notify admin.
- Important announcement or service interruption: admin can send to all retailers, selected retailers, or a relevant audience.
- System delivery failure: log failed push attempts and retry according to policy; keep the in-app notification record.

7.4 Notification delivery and history

- Persist each notification in PostgreSQL with recipient, event type, title/body key or content, related record, created time, read time, delivery state, and language preference.
- Use FCM for push delivery to registered device tokens; store tokens securely and remove invalid tokens.
- Use Supabase Realtime or app refresh for live notification badge and activity updates.
- Support read/unread state and deep links to the relevant order/payment/delivery/invoice/settings page.
- Use templates/localization for English, Hindi, and Marathi.
- Retries must not create duplicate notification records; use event IDs/idempotency keys.
- Admin can see notification delivery status where useful and manually send announcements.
- Critical financial events should remain visible in-app even if push delivery fails.

7.5 Notification configuration

Admin settings should control whether specific announcement categories are pushed, who receives them, and any non-critical reminder frequency. Core order/payment/delivery notifications must not be silently disabled if doing so would undermine the agreed business workflow. The final notification matrix should be approved before implementation.

8. Billing and Tax Scope

8.1 Current client instruction

Yog Milk has stated that it is not GST-registered and that the dairy products covered by this project do not attract GST. Accordingly, the initial scope is a professional non-GST commercial invoice/bill, not a GST tax-invoice module.

The system must not invent or print a GSTIN, GST rates, CGST, SGST, IGST, HSN tax calculations, or a GST-registered supplier representation. The client should have its accountant confirm the applicable legal invoice wording and tax treatment for the actual products and transactions before production. If the business later becomes GST-registered or any product/transaction requires different treatment, that is a formal scope/configuration change.

8.2 Non-GST invoice contents

- Yog Milk brand/business name, logo, contact and business address as supplied by the client.
- Unique invoice number and invoice date.
- Unique order code and order date.
- Retailer/shop name, contact details, and billing/delivery address as applicable.
- Product name/description, pack size, quantity, unit price, line total.
- Subtotal, approved discount, delivery fee if applicable, final total.
- Payment method, payment status, amount received, balance due.
- Invoice terms or notes approved by the business.
- Clear commercial bill/invoice label consistent with accountant-approved requirements; no false GST tax invoice claim.

8.3 Invoice integrity

Invoice records must preserve a snapshot of the billed products, quantities, prices, retailer details, totals, and invoice number. If a billed order needs correction, use a controlled correction/cancellation process and retain history rather than silently rewriting an issued invoice. PDFs should be stored privately and served only to the retailer who owns the invoice or authorized admins.

9. Retailer History, Analytics and Auditability

9.1 Retailer 360° history

The admin must be able to inspect a retailer's complete business relationship in one place: profile and account changes, all orders, order-item details, order status timeline, payments and failed attempts, COD collection records, invoices, delivery records, notifications, and purchasing analytics.

9.2 Analytics

- Retailer: orders count, monthly purchase value, average order value, frequently purchased products, last order, paid total, pending amount, delivered count, and order frequency.
- Admin: order volume, order value, collection total, pending amount, order status distribution, delivery completion/failure, top products, top retailers, and trends by day/week/month/custom range.
- Reports must distinguish order value/sales from money actually collected and outstanding balances.
- Analytics should be calculated from trusted database records/views rather than manually maintained duplicate totals.

9.3 Audit log

Record who did what, when, to which business record, and the outcome. Audit at minimum: retailer approval/suspension, product and price changes, order status updates, payment marked paid manually, payment corrections/refunds, delivery status updates, invoice generation/correction, ordering-window changes, notification broadcasts, and permission changes. Audit logs should be append-only for normal users and should not store secrets or unnecessary sensitive data.

10. Proposed Technical Architecture

10.1 High-level structure

Retailer Flutter app and admin web dashboard both use the same Supabase-backed business data model. Supabase Auth handles identity; PostgreSQL is the source of truth; backend functions perform privileged operations and payment verification; Storage holds product images and private invoice PDFs; Realtime refreshes relevant screens; FCM sends push notifications.

10.2 Trusted backend operations

- Create order and calculate prices/totals.
- Validate ordering window and pending-payment restriction.
- Create Razorpay order and verify payment signatures/webhooks.
- Record COD/offline payment after an authorized admin action.
- Apply valid order/payment/delivery status transitions.
- Generate invoice number and invoice snapshot/PDF.
- Create notification events and route them to correct recipients.
- Generate unique order/delivery/invoice references.
- Write audit records for privileged actions.

10.3 Recommended components

- Flutter: feature-first modules for auth, home, catalogue, cart, orders, payments, delivery, invoices, notifications, profile, localization.
- Admin web: modules for dashboard, retailers, products, schedules, orders, payments, deliveries, invoices, notifications, reports, settings, and audit logs.
- Supabase: Auth, PostgreSQL, Storage, Realtime, Row Level Security (RLS), database functions/triggers where suitable, and Edge Functions or another trusted server environment for privileged integrations.
- Razorpay: server-created payment orders, signature verification, webhooks, idempotent processing, and reconciliation records.

- FCM: push delivery to retailer Android devices and, if supported by the chosen web implementation, admin browser notifications.

Never place Supabase service-role keys, Razorpay secrets, webhook secrets, or other privileged credentials in Flutter code or browser JavaScript.

11. Proposed Database Design

This is the logical schema baseline. Exact columns, indexes, constraints, and migrations should be finalized during technical design. Credit-related tables are deliberately excluded.

11.1 Identity and access

- profiles — user ID linked to Supabase Auth; name, phone, email, preferred language, status, timestamps.
- retailers — retailer/business profile, shop name, owner/contact details, account approval state, billing/delivery details.
- retailer_addresses — one or more addresses if multi-address support is required.
- admin_users or user_roles — administrator identity and permission role; never trust a client-side role flag alone.

11.2 Catalogue

- categories — category name/translation keys, image, sort order, active flag.
- products — product name/description, category, pack size, unit, image, availability, minimum quantity, quantity increment, active flag.
- product_prices — price history/effective dates if pricing changes need to be tracked.
- retailer_product_prices — optional retailer-specific pricing if approved.
- product_announcements — new arrival/availability announcement audience, language/content, active period.

11.3 Orders

- orders — internal ID, unique order_code, retailer_id, order status, payment method, payment status/balance summary, delivery address snapshot, totals, timestamps.
- order_items — order ID, product ID if retained, product name/pack-size snapshot, unit-price snapshot, quantity, line total.
- order_status_history — order ID, previous/new status, actor, timestamp, reason/note.
- business_settings / ordering_schedule — configurable recurring opening/closing times, applicable weekdays, timezone, scheduled exception dates, and closure message. The system calculates OPEN/CLOSED status automatically from this configuration; no manual master switch is provided.
- carts and cart_items — optional persisted cart if needed; the backend still validates all values at checkout.

11.4 Payments

- payments — payment record linked to retailer/order, method, amount, status, reference, recorded/verified timestamp, actor/source.
- payment_attempts or payment_transactions — each Razorpay attempt/reference and its individual state.
- payment_webhook_events — provider event ID, received time, verification result, processed state for idempotency.
- refunds — refund amount, reference, status, reason, and actor where refunds are supported.

A failed attempt must not overwrite a prior successful payment. Use transaction-level records plus a reliable order balance calculation. Manual COD payments must be distinguishable from gateway-confirmed payments.

11.5 Delivery

- deliveries — unique delivery_code, order_id, current delivery status, assignment/dispatch metadata if used, notes.
- delivery_status_history — delivery ID, previous/new status, actor, timestamp, notes/failure reason.

11.6 Billing

- invoices — unique invoice_number, order_id, retailer_id, invoice date, billed totals, amount paid, amount due, status, private PDF storage path, immutable item/business snapshots.
- invoice_items — product/description snapshot, quantity, unit price, line total.
- invoice_events — invoice generated/downloaded/corrected/cancelled events where the business requires history.

11.7 Notifications and audit

- notifications — recipient, event type, localized title/body or template key, related record, created/read timestamps, delivery state.
- notification_deliveries — push provider result, attempt count, error/result metadata, timestamps.
- device_tokens — user/device token, platform, last-seen time, validity state.
- notification_preferences — user preferences for optional/non-critical announcements.
- audit_logs — actor, action, entity type/ID, before/after summary, reason, timestamp, request/event ID.

11.8 Database constraints and indexing

- Unique constraints for order_code, delivery_code, invoice_number, and provider event IDs.
- Foreign keys between retailers, orders, order items, payments, deliveries, invoices, notifications, and history records.
- Indexes for retailer ID, order/payment/delivery status, created timestamps, invoice number, order code, and provider references.
- Use database transactions or trusted backend operations to avoid creating an order without its items, code, initial status history, and notification event.
- Use numeric/decimal database types for currency, not floating-point values.
- Store timestamps consistently and render them in the configured business/user timezone.

12. Security, Privacy and Reliability

- Enable Row Level Security on exposed Supabase tables and write policies that restrict retailers to their own profile, orders, payments, invoices, deliveries, and notifications.
- Admin access must be validated through trusted roles/claims and backend authorization, not by hiding UI controls.
- Privileged operations (price calculation, order creation, manual payment, status transition, invoice generation, notification broadcasts) must be authorized and validated server-side.
- Store invoice PDFs and other private files in non-public storage buckets; use authorized access or short-lived signed links.
- Validate every input, amount, product, quantity, status transition, and reference on the backend.

- Use idempotency for payment webhooks and notification processing.
- Keep audit history for financial and administrative actions.
- Do not log passwords, access tokens, payment secrets, or unnecessary personal data.
- Add backup/restore and operational recovery procedures before launch.
- Test account suspension, unauthorized record access, duplicate webhook delivery, network failure, stale carts, and simultaneous checkout attempts.

PHASE 2 — DEVELOPMENT EXECUTION & IMPLEMENTATION PLANNING

13. Development Strategy

Build in controlled, testable stages. Do not ask a coding agent to build the entire platform in one uncontrolled operation. Each stage must have acceptance criteria, a review checkpoint, and regression testing. Preserve approved business rules and existing working functionality unless a change is explicitly approved.

Stage 0 — Scope freeze and project setup

28. Confirm this revised document with the client.
29. Record the explicit removal of credit functionality in the scope checklist.
30. Confirm ordering hours, payment blocking rule, COD availability, status wording, invoice format, retailer approval process, and product pricing rules.
31. Create repositories, environments, secrets plan, branching strategy, and issue/task board.
32. Establish development, staging, and production Supabase projects; do not use production data for routine testing.

Stage 1 — UX flows and design system

33. Map navigation and screen-level acceptance criteria for retailer app and admin dashboard.
34. Design responsive screens, forms, tables, filters, order details, status timelines, payment history, invoice preview, notification center, and ordering settings.
35. Create design tokens, typography, spacing, accessible colors, buttons, loading/error/empty states, confirmation dialogs, and form validation.
36. Build localization keys for English, Hindi, and Marathi from the beginning.
37. Review the main journeys with the client before connecting production data.

Stage 2 — Database and security foundation

38. Implement schema migrations, foreign keys, unique constraints, indexes, status enums/check constraints, and seed data for development.
39. Configure Supabase Auth with Google Sign-In for Android and web.
40. Implement profiles, retailer onboarding/approval, roles, and RLS policies.
41. Create private storage buckets for product images as appropriate and invoice PDFs.
42. Write automated checks proving that one retailer cannot access another retailer's records and that non-admin users cannot perform admin actions.

Stage 3 — Product catalogue and pricing

43. Implement admin category/product management and image upload.

44. Implement price, pack size, availability, minimum quantity, and increments.
45. Implement retailer catalogue search/filter and cart interactions.
46. Add optional retailer-specific pricing only if the client approves it.
47. Test price changes while a cart is open and ensure checkout uses the backend's current validated price.

Stage 4 — Ordering schedule

48. Build admin controls to configure and edit recurring opening/closing times, applicable weekdays, and scheduled exceptions/closures. Ordering status must update automatically from the saved schedule.
49. Show the current status and next opening information in the retailer app.
50. Enforce the schedule in the trusted order-creation operation.
51. Test exact opening/closing boundaries, timezone, schedule edits, scheduled holiday/exception closures, and checkout attempts outside the permitted interval.

Stage 5 — Orders and unique codes

52. Implement backend order creation and server-side total calculation.
53. Generate unique order codes and save order/item snapshots atomically.
54. Implement retailer order list/detail and admin order queue/detail.
55. Implement valid order status transitions and order status history.
56. Implement quick add and reorder using current prices and availability.
57. Test duplicate submissions, network retries, empty carts, unavailable products, and concurrent order attempts.

Stage 6 — Payment restriction and payment methods

58. Implement Razorpay order creation on the trusted backend, payment signature verification, webhook processing, and idempotency.
59. Implement COD checkout with payment status PENDING.
60. Build admin manual payment recording for COD/offline payments with permissions, required fields, confirmation, and audit history.
61. Implement the rule blocking a retailer from placing another order while an unresolved payment balance exists.
62. Ensure verified online payment or authorized recorded offline collection resolves the pending balance; a failed payment attempt does not.
63. Test duplicate webhooks, delayed webhooks, payment failure, admin double-click, partial amount scenarios if allowed, and reconciliation.

Stage 7 — Delivery management

64. Create a delivery record and unique delivery code/reference linked to each applicable order.
65. Build admin delivery queue and individual status updates.
66. Record each transition in delivery_status_history with actor/time/notes.
67. Show delivery timeline and status in the retailer app.
68. Test failed delivery, incorrect transitions, and status updates made by unauthorized users.

Stage 8 — Invoices and billing

69. Confirm the non-GST invoice wording and required business details with Yog Milk and its accountant.
70. Generate unique invoice numbers and immutable invoice/item snapshots.
71. Generate professional PDF bills and store them privately.
72. Expose authorized invoice preview/download to retailers and admin.
73. Link payments to the order/invoice and display amount paid and outstanding.
74. Test invoice totals against order totals and prevent silent edits to issued invoices.

Stage 9 — Notification engine

75. Define and approve the notification event matrix.
76. Create durable notification records from trusted business events.
77. Implement in-app notification center, unread counts, realtime refresh, FCM push, and deep links.
78. Implement localized English/Hindi/Marathi templates.
79. Route admin operational notifications and retailer-specific notifications correctly.
80. Add delivery retries, invalid-token handling, deduplication, and notification audit/diagnostics.
81. Test every critical event with both a retailer account and admin account.

Stage 10 — Retailer history, analytics and reports

82. Build retailer 360° profile and activity timeline.
83. Build retailer purchase, payment, order, delivery, and invoice history.
84. Build admin charts and date-filtered reports using backend queries/views.
85. Separate sales/order totals from payment collection and pending balances.
86. Implement export only for authorized admin roles and ensure exported data is appropriately protected.

Stage 11 — Localization and polish

87. Complete English, Hindi, and Marathi strings and verify UI layouts with longer translated text.
88. Localize date/time, currency formatting, notifications, empty states, validation, and error messages.
89. Ensure product/business names and database records are not accidentally translated unless translations are explicitly provided.
90. Verify the app on physical Android devices and test the admin dashboard at desktop, tablet, and mobile breakpoints.

Stage 12 — QA, security and production release

91. Run end-to-end tests for Google Sign-In, retailer approval, ordering hours, bulk checkout, Razorpay, COD, payment blocking, manual payment confirmation, delivery updates, invoices, notifications, analytics, and history.
92. Perform RLS/security tests and review all privileged backend functions.
93. Verify secrets are server-side only and production environment variables are configured correctly.
94. Test push notifications on physical devices and verify notification permissions and token lifecycle.
95. Test backups, recovery, logging, error handling, and production monitoring.
96. Obtain client acceptance for all key workflows and deliver a deployment/handover guide.

14. Acceptance Criteria

- A retailer can authenticate with Google, complete required onboarding, and only use permitted features for their account status.
- Products and prices managed by admin appear correctly in the retailer app.
- Orders can be placed only while ordering is open and when the retailer has no unresolved payment block.
- Each order has a unique order code and a complete item/price snapshot.
- COD order is created with payment PENDING and cannot be marked PAID by the retailer.
- Only an authorized admin can record COD/offline payment; the action creates a payment record and audit log.
- Razorpay payments are marked paid only after trusted verification.
- A retailer with an unresolved pending payment cannot place a new order; the block clears only after the balance is resolved according to the approved rules.
- Each delivery has a unique delivery reference and a full status history.
- Every meaningful order/payment/delivery/product-arrival/schedule event creates the required notifications for the correct retailer and admin recipients.
- Retailers cannot access another retailer's private records.
- Non-GST invoices have unique invoice numbers, correct totals, secure PDF access, and no invented GST fields.
- Admin can review a retailer's complete order, payment, delivery, invoice, and activity history.
- All sensitive admin actions are auditable.
- English, Hindi, and Marathi are available across the agreed user-facing screens and notification templates.
- No credit allocation, credit balance, credit expiry, credit-based order, or seven-day credit workflow exists in the initial release.

15. Decisions to Confirm Before Implementation

The major business direction is now defined. The following smaller details still need explicit confirmation before they are encoded in production:

97. Exact ordering schedule: recurring hours and applicable weekdays, holidays or scheduled exception closures, how schedule changes are announced, and the required notification preferences.
98. Order status on successful submission: keep PLACED until admin confirms, or move immediately to PROCESSING as requested; recommended display can show PLACED then PROCESSING when the admin/system takes that action.
99. COD availability: all retailers or only selected retailers; whether COD is available for all order values.
100. Pending payment blocking: whether a cancelled order with a due amount is immediately unblocked after cancellation or only after an admin resolves its balance.
101. Partial payment: whether the initial release supports partial payments or only full payment resolution.
102. Who can mark offline/COD payment paid, and whether proof/reference is required.
103. Invoice numbering format, legal business details, invoice footer, and accountant-approved non-GST wording.
104. Delivery code format and whether one order can ever be split into multiple deliveries.

105. Whether product-specific announcement targeting is needed (all retailers, selected retailers, or by product/category).
106. Whether retailer registration is approval-based and which profile fields are mandatory.
107. Whether pricing is one wholesale price for all retailers or varies by retailer.

16. Project Governance and Coding-Agent Rules

- Inspect the repository and current project structure before changing code.
- Do not replace existing working features or rewrite unrelated logic without explicit approval.
- Implement one approved stage at a time and report files changed, migrations added, tests run, known limitations, and manual steps.
- Do not invent business rules, product data, legal details, tax details, prices, payment outcomes, or client credentials.
- Do not hard-code order hours, payment statuses, prices, or business settings that admins are expected to control.
- Do not trust client-calculated totals, client-side role flags, client-side payment success, or client-side schedule checks.
- Do not implement credit functionality; the credit system has been explicitly removed from this project scope.
- Add tests for every business rule and regression-test the end-to-end order/payment/delivery/notification flow after changes.
- Keep secrets in environment/server-side configuration and never commit them to source control.
- Treat database migrations and financial records carefully; do not destructively alter production data.

17. Recommended Build Order

Scope freeze → UX/screen specification → design system → database and RLS → authentication and retailer onboarding → product catalogue/pricing → ordering schedule → orders and unique codes → Razorpay and COD/payment restriction → delivery management → non-GST invoices → notification engine → retailer history and analytics → localization polish → security/QA → production deployment and handover.

18. Final Project Definition

Yog Milk is a B2B dairy ordering and retailer management platform consisting of a Flutter Android app for retailers and a responsive admin web dashboard. It uses Supabase/PostgreSQL as the system of record, Google Sign-In for authentication, Razorpay for verified online payments, COD with admin-recorded collection, admin-managed delivery updates, unique order and delivery references, configurable ordering hours, non-GST billing based on the client's stated current setup, multilingual support, and a comprehensive notification/history/audit system.

The core operating rule is: every meaningful business event must be recorded reliably, visible to the appropriate user, and traceable later. Retailers receive updates relevant to their own business; administrators receive operational visibility across the platform. No credit system or credit-based payment workflow is part of this version.